Assignment 1

อัลกอริทึมเชิงละโมบ: รถกระบะขนผลไม้ไปตลาดไท (Fractional Knapsack)

วิชา CS441 Algorithm Analysis and Design ภาคการศึกษาที่ 1/2569

ผู้สอน อาจารย์อรุณ เรืองศิลป์กลการ ภาควิชาวิทยาการคอมพิวเตอร์

คณะเทคโนโลยีสารสนเทศและนวัตกรรม มหาวิทยาลัยกรุงเทพ

คะแนนเต็ม	30 คะแนน
ระยะเวลาทำ	7 วัน
การส่งงาน	ไฟล์ Python 1 ไฟล์ ชื่อ A1_Greedy_<รหัสนักศึกษา>.py ที่เติมจาก A1_template.py อัปโหลด MS Teams ไม่รับงานเขียนมือหรือไฟล์รูปแบบอื่นทุกกรณี
บทอ้างอิง	บทที่ 5 หัวข้อ 5.6 (Fractional Knapsack) ข้อ (ง) ใช้สัญกรณ์เชิงเส้นกำกับจากบทที่ 3

1. สถานการณ์

ช่วงปิดเทอมนักศึกษารับงานขับรถกระบะรับซื้อผลไม้จากสวนแห่งหนึ่งในจังหวัดจันทบุรีเพื่อนำไปขายส่งที่ตลาดไท จังหวัดปทุมธานี รถกระบะบรรทุกน้ำหนักได้สูงสุด W กิโลกรัม

สวนมีผลไม้ 5 ชนิด ชนิดที่ i มีปริมาณพร้อมขาย w_i กิโลกรัม หากรับซื้อทั้งล็อตไปขายจะได้กำไร v_i บาท ผลไม้ทุกชนิดชั่งแบ่งขายตามน้ำหนักได้ และกำไรเป็นสัดส่วนโดยตรงกับน้ำหนักที่รับซื้อ กล่าวคือ ถ้ารับซื้อชนิดที่ i มาสัดส่วน x_i ของล็อต ($0 \le x_i \le 1$) จะได้กำไร $v_i x_i$ บาท และใช้พื้นที่บรรทุก $w_i x_i$ กิโลกรัม

เป้าหมาย: เลือกสัดส่วน $x_1, \ldots, x_5$ ที่ทำให้กำไรรวม $\sum v_i x_i$ สูงที่สุด ภายใต้เงื่อนไข $\sum w_i x_i \le W$ และ $0 \le x_i \le 1$ สำหรับทุก i

2. ข้อมูลโจทย์ส่วนตัว

ข้อมูลของแต่ละคนสร้างจาก 5 หลักท้ายของรหัสนักศึกษา เรียกว่า d_1, d_2, d_3, d_4, d_5 (หลักที่เป็น 0 ใช้ค่า 10) ตัวอย่าง รหัส 1350701288 ⟶ 01288 ⟶ $d_1 = 10, d_2 = 1, d_3 = 2, d_4 = 8, d_5 = 8$

นักศึกษาไม่ต้องคำนวณเอง เมื่อกรอก STUDENT_ID ใน BLOCK 1 ของไฟล์ template แล้ว BLOCK 2 จะคำนวณ d_1–d_5 ตารางผลไม้ และความจุรถให้โดยอัตโนมัติตามสูตรต่อไปนี้

รหัส	**ผลไม้**	**น้ำหนักที่สวนมี w_i (กก.)**	**กำไรถ้ารับซื้อทั้งล็อต v_i (บาท)**
A	ทุเรียน	$10 \times d_2$	$100 \times d_3$
B	มังคุด	$10 \times d_3$	$60 \times d_4$
C	เงาะ	$10 \times d_4$	$40 \times d_5$
D	ลองกอง	$10 \times d_5$	$80 \times d_1$
E	มะม่วง	$10 \times d_1$	$50 \times d_2$

ความจุรถกระบะ $W = 5 \times (d_1 + d_2 + d_3 + d_4 + d_5)$ กิโลกรัม
(เท่ากับครึ่งหนึ่งของน้ำหนักผลไม้ทั้งหมดที่สวนมี)

กติกาเมื่ออัตรากำไรต่อกิโลกรัมเท่ากัน ให้ชนิดที่มีรหัสตัวอักษรมาก่อนอยู่ก่อน (A ก่อน B, B ก่อน C, ...)

ตัวเลขทุกค่าให้ใช้ fractions.Fraction เพื่อความแม่นยำ ห้ามปัดเศษ
(โปรแกรมตรวจเทียบค่าแบบเศษส่วนตรงตัว)

3. โครงสร้างไฟล์ A1_template.py

ไฟล์แบ่งเป็น 7 บล็อก นักศึกษาแก้ไขได้เฉพาะบล็อกที่ระบุ

บล็อก	**เนื้อหา**	**นักศึกษาทำอะไร**
BLOCK 1	ข้อมูลนักศึกษา: STUDENT_ID, STUDENT_NAME, COURSE, SECTION, ASSIGNMENT	กรอกข้อมูลของตนเอง รหัสต้องเป็น 10 หลักที่ถูกต้อง
BLOCK 2	คำนวณ d_1–d_5 สร้างตารางผลไม้และ W จาก STUDENT_ID	ห้ามแก้ไข
BLOCK 3	ข้อ (ก) ฟังก์ชัน profit_rate, sort_by_rate	เติมโค้ดแทน TODO
BLOCK 4	ข้อ (ข) ฟังก์ชัน fractional_knapsack	เติมโค้ดแทน TODO

บล็อก	เนื้อหา	นักศึกษาทำอะไร
BLOCK 5	ข้อ (ค) ฟังก์ชัน knapsack_one_indivisible	เติมโค้ดแทน TODO
BLOCK 6	ข้อ (ง) ตัวแปร BIG_O และ BIG_O_REASON	เติมคำตอบแทน O(?)
BLOCK 7	main รันทุกข้อและพิมพ์ผลลัพธ์รูปแบบมาตรฐาน	ห้ามแก้ไข

4. คำถาม (รวม 30 คะแนน)

(ก) จัดเตรียมข้อมูล [6 คะแนน]

(ก.1) ตรวจสอบว่า BLOCK 2 สร้างตารางผลไม้และ W ของตนเองถูกต้อง (คะแนนส่วนนี้ให้เมื่อ STUDENT_ID ถูกต้องและโปรแกรมรันได้) [2 คะแนน]

(ก.2) เขียน profit_rate(item) คืนอัตรากำไรต่อกิโลกรัม $r_i = v_i / w_i$ เป็น Fraction [2 คะแนน]

(ก.3) เขียน sort_by_rate(items) คืน list ใหม่ที่เรียงตาม r_i จากมากไปน้อย ใช้กติกาตัวอักษรเมื่อเท่ากัน และห้ามแก้ไข list เดิม [2 คะแนน]

(ข) Fractional Knapsack [10.5 คะแนน]

เขียน fractional_knapsack(items, W) ตามอัลกอริทึมในหัวข้อ 5.6: รับซื้อผลไม้ตามลำดับ r_i ทั้งล็อตไปเรื่อย ๆ จนพบล็อตแรกที่ใส่ไม่พอ จึงรับซื้อบางส่วนให้รถเต็มพอดีแล้วหยุด ฟังก์ชันต้องคืน dict ที่มี

(ข.1) plan: ลำดับการรับซื้อทีละขั้น แต่ละขั้นมี code, take (กก.), profit (บาท), remain (กก. ที่เหลือของรถ) [5 คะแนน]

(ข.2) x: สัดส่วนที่รับซื้อของทั้ง 5 ชนิด และ split: รหัสชนิดที่ถูกรับซื้อบางส่วน (หรือ None) [2.5 คะแนน]

(ข.3) total: กำไรรวม โปรแกรมตรวจจะคำนวณน้ำหนักรวมจาก x และต้องเท่ากับ W พอดี [3 คะแนน]

(ค) เงื่อนไขจริงจากสวน: ล็อตที่แบ่งไม่ได้ [7.5 คะแนน]

เจ้าของสวนแจ้งว่า ผลไม้ชนิดที่ถูกรับซื้อเพียงบางส่วนในข้อ (ข) (เรียกว่าชนิด k) ต้องขายทั้งล็อตเท่านั้น ส่วนอีก 4 ชนิดยังแบ่งได้ตามเดิม (BLOCK 7 จะหา k ให้จากผลข้อ (ข) โดยอัตโนมัติ หากไม่มีชนิดที่ถูกแบ่งจะใช้ชนิดถัดไปในลำดับที่ยังไม่ได้รับซื้อ)

เขียน knapsack_one_indivisible(items, W, k) ที่พิจารณา 2 กรณีแล้วเลือกกรณีที่กำไรสูงกว่า

(ค.1) กรณี out ไม่รับซื้อ k เลย: รันอัลกอริทึมเชิงละโมบกับ 4 ชนิดที่เหลือด้วยความจุ W [2.5 คะแนน]

(ค.2) กรณี in รับซื้อ k ทั้งล็อต: ได้กำไร v_k ความจุเหลือ W − w_k แล้วรันอัลกอริทึมเชิงละโมบกับ 4 ชนิดที่เหลือ (ถ้า w_k > W ให้กำไรกรณีนี้เป็น −1) [2.5 คะแนน]

(ค.3) best: "out" หรือ "in" (เท่ากันให้ตอบ "out") และ best_total: กำไรของกรณีที่เลือก [2.5 คะแนน]

ข้อสังเกตที่ควรได้จากข้อนี้: การใช้อัลกอริทึมเชิงละโมบแบบเดิมแล้ว "ตัดเศษส่วนทิ้ง" เท่ากับทำกรณี out เสมอโดยไม่เคยพิจารณากรณี in ซึ่งในข้อมูลของนักศึกษาบางคนกรณี in ให้กำไรมากกว่า แสดงว่าเมื่อมีข้อจำกัดแบบแบ่งไม่ได้แม้เพียงชนิดเดียว การตัดสินใจเฉพาะหน้าไม่รับประกันคำตอบที่ดีที่สุดอีกต่อไป

(ง) สัญกรณ์เชิงเส้นกำกับ [6 คะแนน]

ให้ n เป็นจำนวนชนิดผลไม้ เติม Big-O ที่แน่นที่สุดของโค้ดที่ตนเองเขียนลงในตัวแปร BIG_O ทั้ง 5 รายการ (รายการละ 1.2 คะแนน) ได้แก่ การคำนวณอัตรากำไรของทุกชนิด, การเรียงลำดับ, วงวนรับซื้อ, ฟังก์ชัน (ข) ทั้งหมด และฟังก์ชัน (ค) ทั้งหมด รูปแบบคำตอบ เช่น "O(n)", "O(n log n)", "O(n²)" และเขียนเหตุผลสั้น ๆ ใน BIG_O_REASON ว่าขั้นตอนใดเป็นตัวกำหนดเวลาทำงาน และเหตุใด (ค) จึงไม่ช้ากว่า (ข) ในเชิงเส้นกำกับ

5. ตัวอย่างผลลัพธ์ที่ถูกต้อง (รหัสตัวอย่าง 1350701288 ⟶ d = 10, 1, 2, 8, 8)

เมื่อเติมโค้ดครบและรัน A1_Greedy_1350701288.py โปรแกรมจะพิมพ์ผลลัพธ์ดังนี้ (ตัดบรรทัด INFO และ A1 ออก) นักศึกษาใช้เทียบรูปแบบได้ แต่ตัวเลขของตนเองจะต่างจากนี้

```
[D] d1=10 d2=1 d3=2 d4=8 d5=8
[W] 145
[A2] A r=20
[A2] B r=24
[A2] C r=4
[A2] D r=10
[A2] E r=1/2
[A3] order=B,A,D,C,E
[B1] step=1 code=B take=20 profit=480 remain=125
[B1] step=2 code=A take=10 profit=200 remain=115
[B1] step=3 code=D take=80 profit=800 remain=35
[B1] step=4 code=C take=35 profit=140 remain=0
[B2] x A=1 B=1 C=7/16 D=1 E=0
[B2] split=C
[B3] total=1620 weight=145
[C0] k=C
[C1] case_out=2995/2
[C2] case_in=1350
[C3] best=out best_total=2995/2
[D] profit_rate_all=O(n)
[D] sort_by_rate=O(n log n)
[D] take_loop=O(n)
[D] fractional_knapsack=O(n log n)
[D] knapsack_one_indivisible=O(n log n)
```

อ่านผล: รับซื้อมังคุด ทุเรียน ลองกอง ทั้งล็อต แล้วรับซื้อเงาะ 35 กก. จาก 80 กก. (7/16 ของล็อต) ได้กำไร 1,620 บาท เมื่อเงาะต้องขายทั้งล็อต กรณีไม่รับซื้อเงาะได้ 1,497.50 บาท กรณีรับซื้อทั้งล็อตได้ 1,350 บาท จึงเลือกไม่รับซื้อ

6. ข้อกำหนดการส่งงาน

- ส่งไฟล์ Python ไฟล์เดียว ชื่อ A1_Greedy_<รหัสนักศึกษา>.py เช่น A1_Greedy_1350701288.py รหัสในชื่อไฟล์ต้องตรงกับ STUDENT_ID ในไฟล์ โปรแกรมตรวจจะยึดรหัสในชื่อไฟล์เป็นหลัก
- ไฟล์ต้องรันได้ด้วย python3 โดยไม่มี error และพิมพ์ผลลัพธ์ตามรูปแบบของ BLOCK 7 ไฟล์ที่รันไม่ได้หรือค้างเกิน 20 วินาทีจะได้ 0 คะแนนในข้อที่ไม่มีผลลัพธ์
- ห้ามแก้ไข BLOCK 2 และ BLOCK 7 ห้ามเปลี่ยนชื่อหรือพารามิเตอร์ของฟังก์ชันที่กำหนด ห้ามใช้ library ภายนอก

- ไม่รับงานเขียนมือ ภาพถ่าย PDF หรือไฟล์รูปแบบอื่นทุกกรณี
- ส่งช้า หักวันละ 10% ของคะแนนที่ได้ ไม่รับงานหลังเลย 3 วัน

7. การให้คะแนน

ระดับ	ข้อ	คะแนน	สัดส่วน	ลักษณะ
ง่าย	(ก)	6	20%	กรอกข้อมูลถูก คำนวณอัตรากำไร และเรียงลำดับ
ปานกลาง	(ข)	10.5	35%	implement อัลกอริทึมตามตำราให้ผลถูกทุกขั้น
ยาก	(ค)	7.5	25%	ปรับใช้กับเงื่อนไขจริงที่ตำราไม่ได้ให้สูตรตรง ๆ
ยากมาก	(ง)	6	20%	วิเคราะห์สัญกรณ์เชิงเส้นกำกับของโค้ดตนเอง (บทที่ 3)

ข้อ (ข.1) ให้คะแนนตามสัดส่วนขั้นที่ถูกต้อง ข้อ (ค) จะได้คะแนนเมื่อชนิด k ที่ได้จากข้อ (ข) ถูกต้อง ดังนั้นควรทำข้อ (ข) ให้ถูกก่อน

8. นโยบายความซื่อสัตย์ทางการศึกษา

ค้นคว้าจากตำรา เอกสารประกอบการสอน และแหล่งออนไลน์เพื่อทำความเข้าใจได้
แต่โค้ดต้องเขียนด้วยตนเอง ห้ามคัดลอกจากเพื่อน จากเครื่องมือปัญญาประดิษฐ์ หรือจากแหล่งใด ๆ
งานที่คล้ายกันผิดปกติหรือส่งไฟล์ที่รหัสไม่ตรงกับผู้ส่งจะได้ 0 คะแนน